

De Lento a Fulminante

Dominando la Concurrency en Java y Spring Boot



La Tiranía de lo Secuencial: ¿Por qué no podemos permitirnos esperar?

Un programa secuencial es como una autopista de un solo carril. Cada tarea (coche) debe esperar a que la anterior termine. Si una tarea es lenta, todo el sistema se detiene.

En el mundo de las aplicaciones modernas, esto se traduce en usuarios frustrados y sistemas ineficientes.

“Normalmente en tus programas Java solo se utiliza un thread, pero tienes la capacidad de crear varios y realizar múltiples tareas al mismo tiempo.”



El Problema en Código: 9 Segundos de Espera Inaceptable

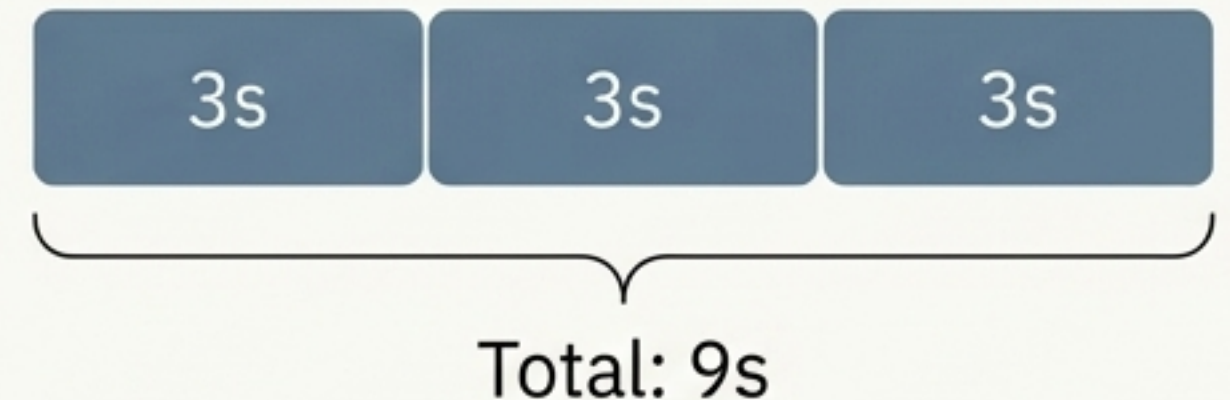
Analicemos un caso simple: ejecutar tres tareas, cada una con una duración de 3 segundos. De forma síncrona, el resultado es predecible y dolorosamente lento.

Ejecución Síncrona

```
public void ejecutarTareasSync() {  
    long startTime = System.currentTimeMillis();  
    tareaLentaSync("Tarea A");  
    tareaLentaSync("Tarea B");  
    tareaLentaSync("Tarea C");  
    long endTime = System.currentTimeMillis();  
    System.out.println("Tiempo Total: " +  
        (endTime - startTime) / 1000.0 + " segundos");  
}  
  
private void tareaLentaSync(String nombre) {  
    try {  
        Thread.sleep(3000); // Simula trabajo de 3 segundos  
    } catch (InterruptedException e) {  
        Thread.currentThread().interrupt();  
    }  
}
```

Resultado

≈ 9
segundos



La Solución Mágica: `@Async` al Rescate

Con una simple anotación, Spring Boot nos permite ejecutar estas tareas en paralelo. El framework se encarga de la gestión de hilos, permitiéndonos enfocarnos en la lógica de negocio.

Ejecución Asíncrona con @Async

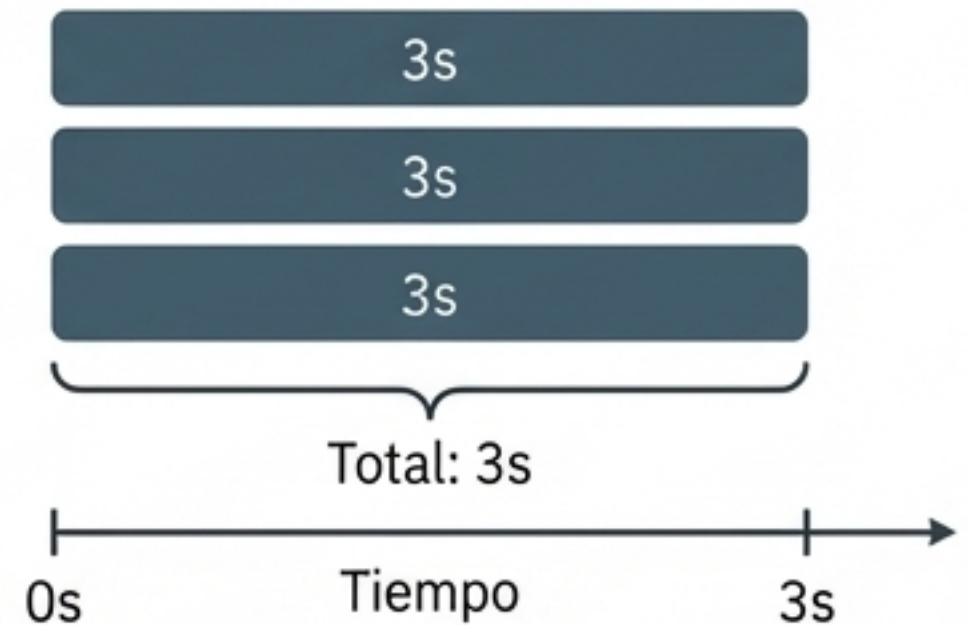
```
// En el método principal...
CompletableFuture<Void> tareaA = service.tareaLentaAsync("Tarea A");
CompletableFuture<Void> tareaB = service.tareaLentaAsync("Tarea B");
CompletableFuture<Void> tareaC = service.tareaLentaAsync("Tarea C");

// Esperar a que todas terminen
CompletableFuture.allOf(tareaA, tareaB, tareaC).join();

// En el Service...
@Async("taskExecutor")
public CompletableFuture<Void> tareaLentaAsync(String nombre) {
    // ... misma lógica de 3 segundos de sleep
    return CompletableFuture.completedFuture(null);
}
```

Nuevo Resultado

≈ 3
segundos



¿Qué Sucede Realmente? De `@Async` a los Hilos de Java

`@Async` no es magia. Es una poderosa abstracción sobre el modelo de hilos nativo de Java. En Java, existen dos formas fundamentales de crear hilos:

`extends Thread`

Se hereda directamente de la clase `Thread`.

```
class MiHilo extends Thread {  
    public void run() { /* ... */ }  
}  
MiHilo hilo = new MiHilo();  
hilo.start();
```

- ✗ Limita la herencia. "Sabemos que en Java solo se hereda de una sola clase".

`implements Runnable` (Mejor Práctica)

Se implementa la interfaz `Runnable`. Es más flexible.

```
class MiTarea implements Runnable {  
    public void run() { /* ... */ }  
}  
Thread hilo = new Thread(new MiTarea());  
hilo.start();
```

- ✓ "Nos da la posibilidad de extender de alguna otra clase".

Spring gestiona la creación de `Thread`s y la ejecución de `Runnable`s por nosotros, permitiendo un código más limpio y declarativo.

La Concurrency en Acción: Casos de Uso Reales

El poder de `@Async` va más allá de simples tareas. Permite construir aplicaciones robustas y con alta capacidad de respuesta.

Caso de Uso 1: Reproducción de Películas (Ejercicio 2)

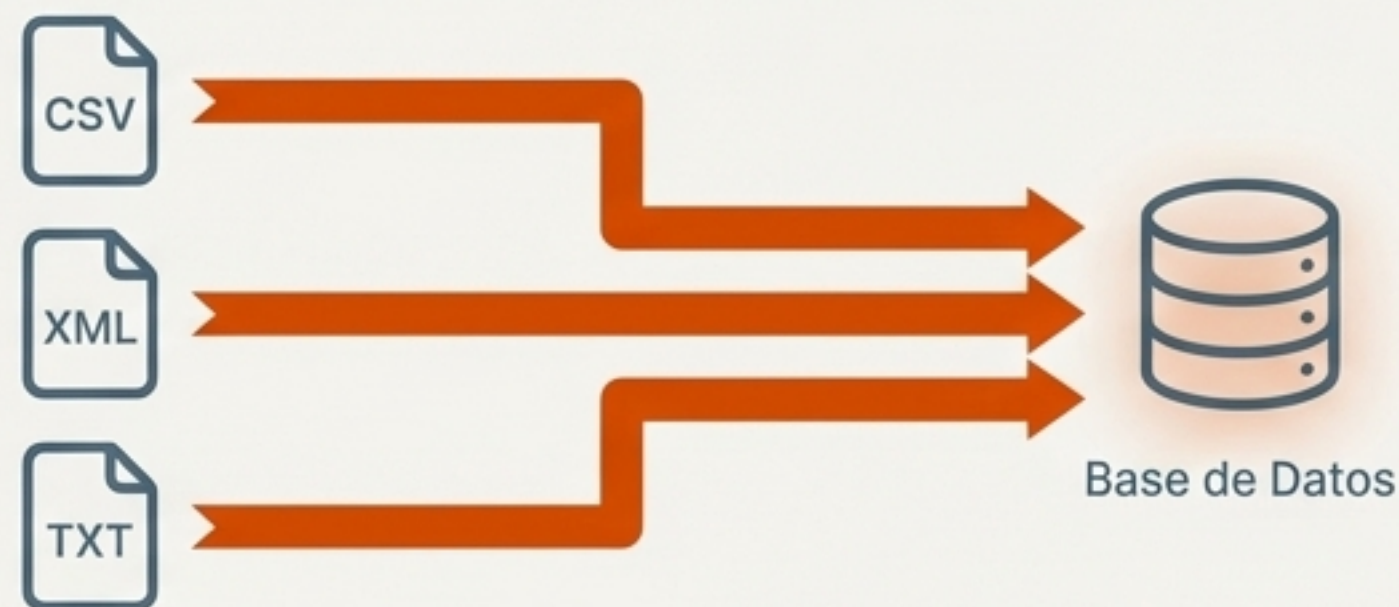
Simular la reproducción simultánea de varias películas, cada una con una duración aleatoria. El sistema no se bloquea esperando a la película más larga.



```
public CompletableFuture<String> reproducir(String titulo)
```

Caso de Uso 2: Importación de Ficheros (Ejercicio 3)

Procesar en paralelo múltiples ficheros de datos (CSV, XML, TXT). El tiempo total de importación se reduce drásticamente al no tener que procesar un fichero tras otro.

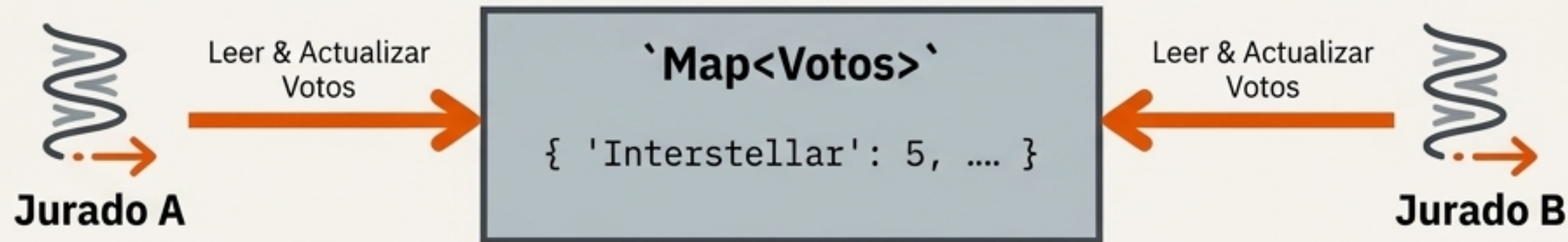


```
// Leer varios ficheros y cargar objetos Pelicula usando @Async
```


El Peligro Oculto: ¿Qué pasa cuando los hilos comparten datos?

Escenario: Votación de los Premios Óscar (Ejercicio 4)

- **Requisito:** Múltiples "jurados" (hilos) deben votar por sus películas favoritas.
- **Mecanismo:** Cada hilo actualiza un ``Map<String, Integer>`` compartido que contiene el recuento de votos.
- **Implementación:** Usamos ``@Async`` para que cada jurado vote de forma concurrente.



Si el Jurado A y el Jurado B leen el contador de votos de "Interstellar" (que es 5) al mismo tiempo, ambos calculan ``5 + 1 = 6``, y ambos escriben ``6``...

¿Cuál es el resultado final? ¿6 o 7?

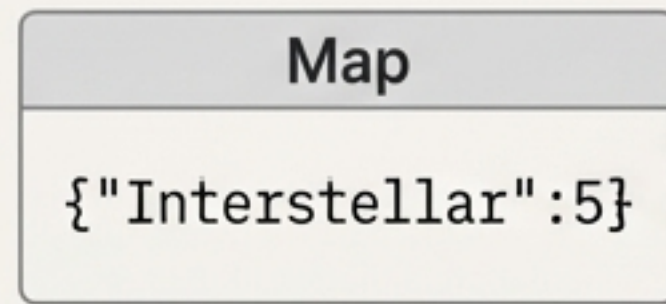
Anatomía de un Desastre: La Condición de Carrera (Race Condition)

"Synchronization is needed to prevent data inconsistency when threads share resources."

Una condición de carrera ocurre cuando el resultado de un sistema depende del orden impredecible en que se ejecutan operaciones sobre un recurso compartido.

Visual: La Sobrescritura Perdida

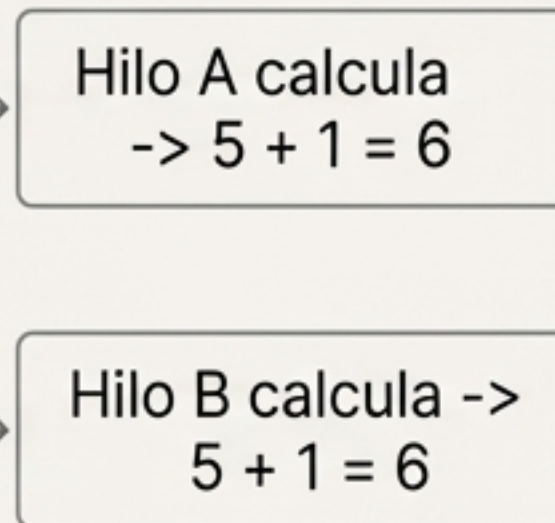
Paso 1: Estado Inicial



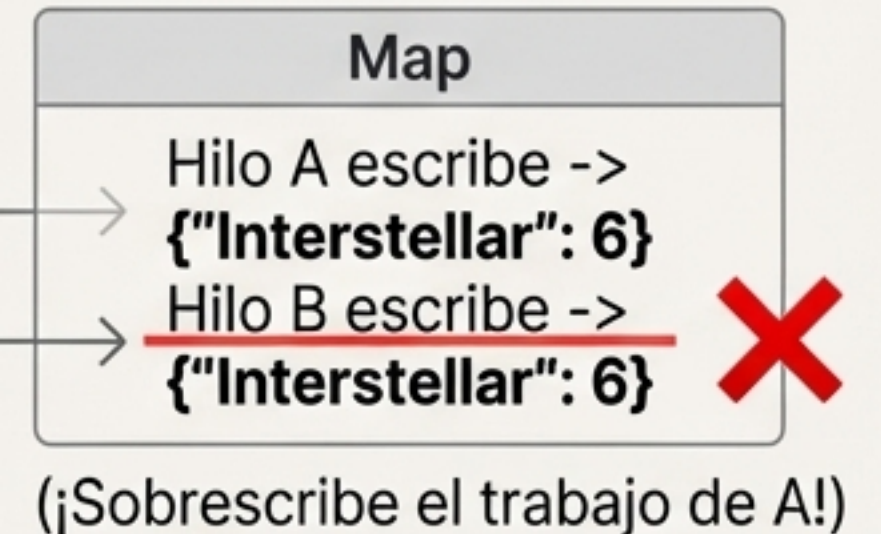
Paso 2: Lectura Simultánea



Paso 3: Cálculo en Paralelo



Paso 4: Escritura Secuencial (pero incorrecta)



¡Hemos perdido un voto! Resultado Final: `{ 'Interstellar': 6 }`.

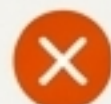
Herramienta 1: Colecciones a Prueba de Hilos

La primera línea de defensa es utilizar estructuras de datos diseñadas específicamente para el acceso concurrente. Para nuestro `Map`, la solución es `ConcurrentHashMap`.

¿Cómo funciona?

`ConcurrentHashMap` utiliza un mecanismo de bloqueo a nivel de 'segmento' o 'bucket' en lugar de bloquear todo el mapa. Esto permite que múltiples hilos escriban en diferentes partes del mapa simultáneamente sin corromper los datos.

Antes (Vulnerable)



```
private Map<String, Integer> votos = new  
HashMap<>(); // Vulnerable a condiciones de  
carrera
```

Después (Seguro)



```
private Map<String, Integer> votos = new  
ConcurrentHashMap<>(); // Hilo-seguro
```

Con un simple cambio de implementación, garantizamos la atomicidad de las operaciones y la integridad de los votos.

Herramienta 2: Control Explícito con `synchronized`

¿Y si el recurso compartido no es una colección? Para proteger cualquier bloque de código (una "sección crítica"), Java nos proporciona la palabra clave `synchronized`.

Concepto Clave

`synchronized` funciona como una cerradura (lock). Solo un hilo puede 'poseer' la cerradura de un objeto a la vez. Cualquier otro hilo que intente entrar en un bloque `synchronized` sobre el mismo objeto deberá esperar a que el primero libere la cerradura.



```
private final Object lock = new Object();
private Map<String, Integer> votos = new HashMap<>();

public void votar(String pelicula) {
    // ...
    synchronized (lock) { // Solo un hilo puede entrar aquí a la vez
        Integer conteoActual = votos.getOrDefault(pelicula, 0);
        votos.put(pelicula, conteoActual + 1);
    } // El lock se libera automáticamente al salir del bloque
    // ...
}
```

Uso

Ideal para proteger operaciones complejas que involucren múltiples pasos sobre datos compartidos.

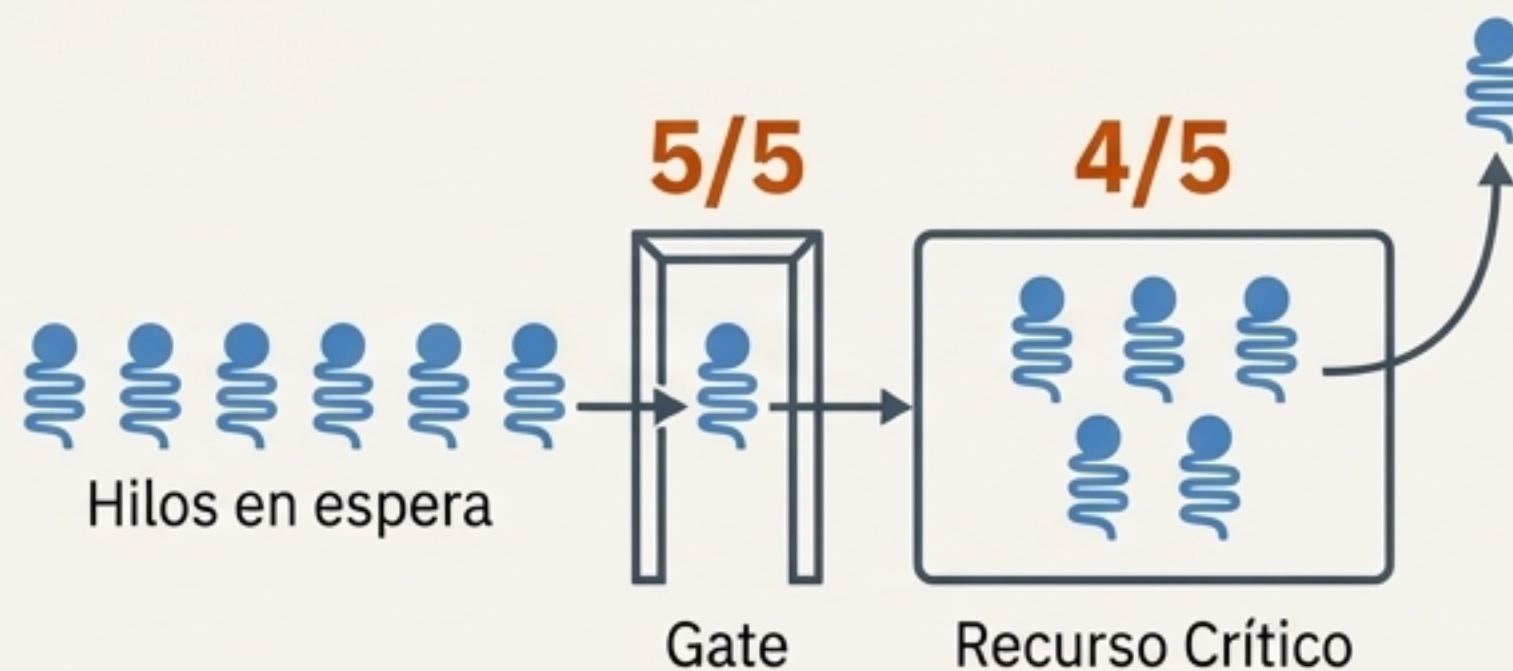
Herramienta 3: Gestión de Recursos con `Semaphore`

La concurrencia no solo se trata de evitar conflictos, sino también de gestionar el acceso limitado a recursos (conexiones a BBDD, APIs externas, etc.). Un `Semaphore` es como un portero de discoteca con un número limitado de pases.

Analogía: "Haz que solo 5 hilos puedan votar al mismo tiempo usando un `Semaphore`."

Mecánica

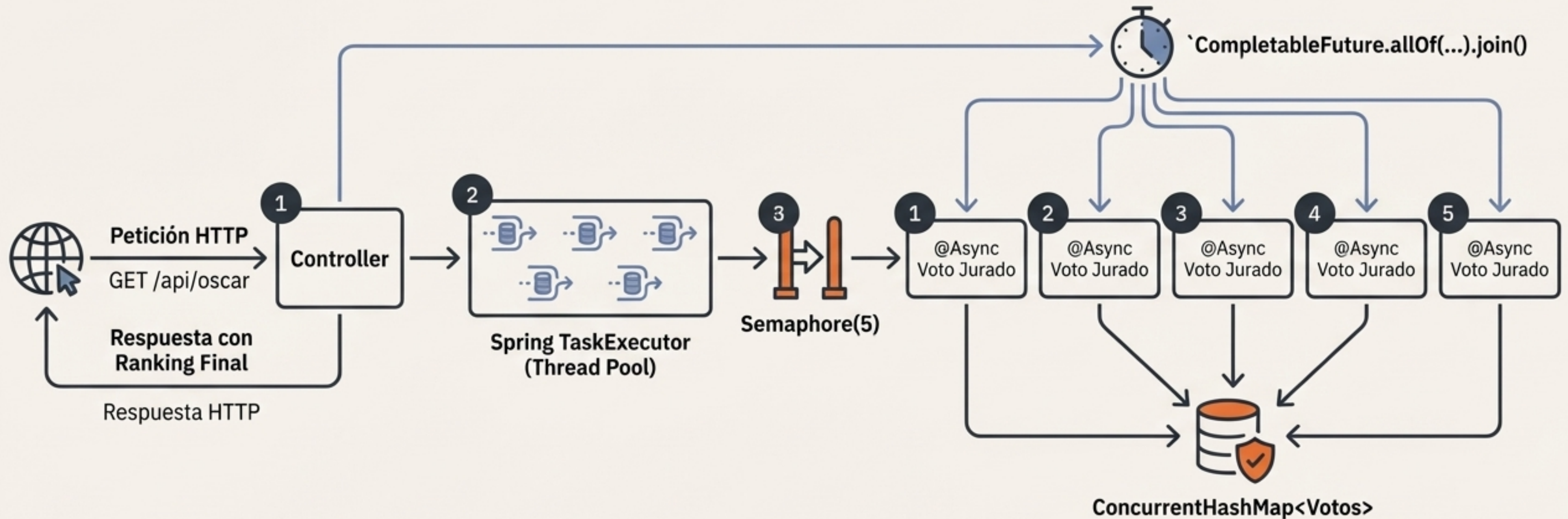
1. `Semaphore sem = new Semaphore(5);`
// Creamos un semáforo con 5 permisos.
2. `sem.acquire();`
// Un hilo pide un permiso. Si no hay disponibles, espera.
3. `// ...código que accede al recurso limitado...`
4. `sem.release();` // El hilo devuelve el permiso para que otro pueda usarlo.



Beneficio

Previene la saturación del sistema, garantizando que no se excedan los límites de los recursos críticos y manteniendo la estabilidad de la aplicación.

El Blueprint de la Maestría: Arquitectura Concurrente Completa



Este diseño combina la simplicidad de `@Async`, la seguridad de las colecciones concurrentes y el control de `Semaphore` para crear una solución rápida, segura y escalable.

La Concurrency es un Superpoder. Úsalo con Sabiduría.

La concurrencia transforma las aplicaciones, llevándolas de lentas y bloqueantes a rápidas y receptivas. Como todo superpoder, debe ser manejado con entendimiento y las herramientas adecuadas.

- **Empieza con un Problema:** Identifica los cuellos de botella secuenciales.
- **Usa Abstracciones:** Prefiere `@Async` y el `Executor Framework` de Spring sobre la gestión manual de hilos.
- **Protege el Estado Compartido:** Utiliza colecciones concurrentes o `synchronized` para evitar condiciones de carrera.
- **Gestiona los Recursos:** Usa `Semaphore` para evitar la sobrecarga del sistema.

